

200 HTTP Status Code in ASP.NET Core Web API

Back to: [ASP.NET Core Web API Tutorials](#)

200 HTTP Status Code in ASP.NET Core Web API

In this article, I will discuss **how to Return a 200 OK HTTP Status Code from the ASP.NET Core Web API Controller Action method** with Examples. Please read our previous article, which overviews the [HTTP status code](#). We will work with the same example we created in our [Controller Action Return Types in the ASP.NET Core Web API](#) article.

200 OK HTTP Status Code:

The 200 HTTP Status Code is known as “OK”. It indicates that the HTTP request made by the client was successfully received, understood, and processed by the server and that the server has returned the expected response. The following are some of the key points about the 200 HTTP status code:

- **General Use:** It's the most common HTTP status code for successful HTTP requests. A 200 status is typically returned when a webpage or resource acts exactly as expected.
- **Contexts of Use:** This status code can be used in response to any request method (GET, POST, PUT, DELETE, etc.) and indicates that the requested action was successfully completed.
- **Response Body:** Responses with a 200 OK status code often include a body that contains the data requested by the client. This could be a webpage, API data, or a file.
- **Implications in RESTful APIs:** In APIs, a 200 OK status code is typically used to communicate that the requested operation was performed successfully, and the response body contains the expected data.

How to Return 200 OK HTTP Status Code in ASP.NET Core Web API?

In ASP.NET Core Web API, returning a 200 HTTP status code can be done in various ways depending on the context and the specific data you want to return to the client. The following are some of the common methods to return a 200 OK response:

Returning Ok():

ASP.NET Core Web API provided the `Ok` method to return the HTTP 200 OK Status Code. When you want to return data, you can pass the data as an argument to this method. This approach automatically serializes the data to JSON (or XML, based on the request's `Accept` header) and sets the content type of the response. If you check the `ControllerBase` class, you will find two overloaded versions of the `Ok` method, as shown below.

```
...public virtual OkResult Ok()...  
...public virtual OkObjectResult Ok([ActionResultObjectValue] object? value)...
```

The first overloaded version, which does not take any input parameter, will create an object that produces an empty HTTP 200 OK Status code as a response. On the other hand, the second overloaded version takes an object (any value) as input and creates an object that produces HTTP 200 OK Status code as a response. The input data it takes will be formatted in the response body (either in JSON or XML format).

Further, if you notice, the first overloaded version of the OK method returns **OkResult**. Now, right-click on the OkResult and choose to go to definition, and you will see the following definition.

```
public class OkResult : StatusCodeResult  
{  
    private const int DefaultStatusCode = StatusCodes.Status200OK;  
    ...public OkResult()...  
}
```

As you can see, the OkResult set the **DefaultStatusCode value to 200**. As you can see in the above image, this method is inherited from the **StatusCodeResult** class. Further, if you right-click on the StatusCodeResult class and choose to go to the definition, then you will find the following definition.

```
public partial class StatusCodeResult : ActionResult, IClientErrorActionResult  
{  
    ...public StatusCodeResult([ActionResultStatusCode] int statusCode)...  
    ...public int StatusCode ...  
    int? IStatusCodeActionResult.StatusCode => StatusCode;  
    ...public override void ExecuteResult(ActionContext context)...  
    private static partial class Log  
    {  
        ...public static partial void HttpStatusCodeResultExecuting(ILogger logger, int statusCode);  
    }  
}
```

The StatusCodeResult class is inherited from **ActionResult**, and we have already discussed that the **ActionResult** class is inherited from the **IActionResult** interface. So, you can use **ActionResult** or **IActionResult** as the return type for the **OK** method. The StatusCodeResult class has the StatusCode property, and using this property, you can set or get the proper status code. It also has the ExecuteResult method, which will set the Status Code of the HTTP Response and return it to the client.

Further, if you notice, the second overloaded version of the OK method returns **OkObjectResult**. Now, right-click on the OkObjectResult class and choose to go to definition, and you will see the following definition.

```
public class OkObjectResult : ObjectResult
{
    private const int DefaultStatusCode = StatusCodes.Status200OK;

    ...public OkObjectResult(object? value)...
}
```

As you can see, the **OkObjectResult** class is inherited from the **ObjectResult** class. Now, again, right-click in the ObjectResult class and choose to go to definition, and you will find the following definition of the ObjectResult class.

```
public class ObjectResult : ActionResult, IStatusCodeActionResult
{
    private MediaTypeCollection _contentTypes;

    ...public ObjectResult(object? value)...

    ...public object? Value ...
    ...public FormatterCollection<IOutputFormatter> Formatters ...
    ...public MediaTypeCollection ContentTypes...
    ...public Type? DeclaredType ...
    ...public int? StatusCode { get; set; }

    ...public override Task ExecuteResultAsync(ActionContext context)...
    ...public virtual void OnFormatting(ActionContext context)...
}
```

If you notice here, the ObjectResult class is inherited from the ActionResult class, and we all know the ActionResult class is inherited from the IActionResult interface. So, we can use ActionResult or IActionResult as the return type for the second overloaded version of the OK method. Further, the ObjectResult class has the StatusCode property, which we can use to set the proper status code, and the ExecuteResultAsync method is used to set the status code in the HTTP Response, and this method is also used to convert the object value to either JSON or XML format and store it in the Response body which will return to the client.

Returning 200 OK HTTP Status Code Without Data:

In this case, we have to use the first overloaded version of the OK method, which does not take any parameters. So, please modify the Employee Controller as shown below.

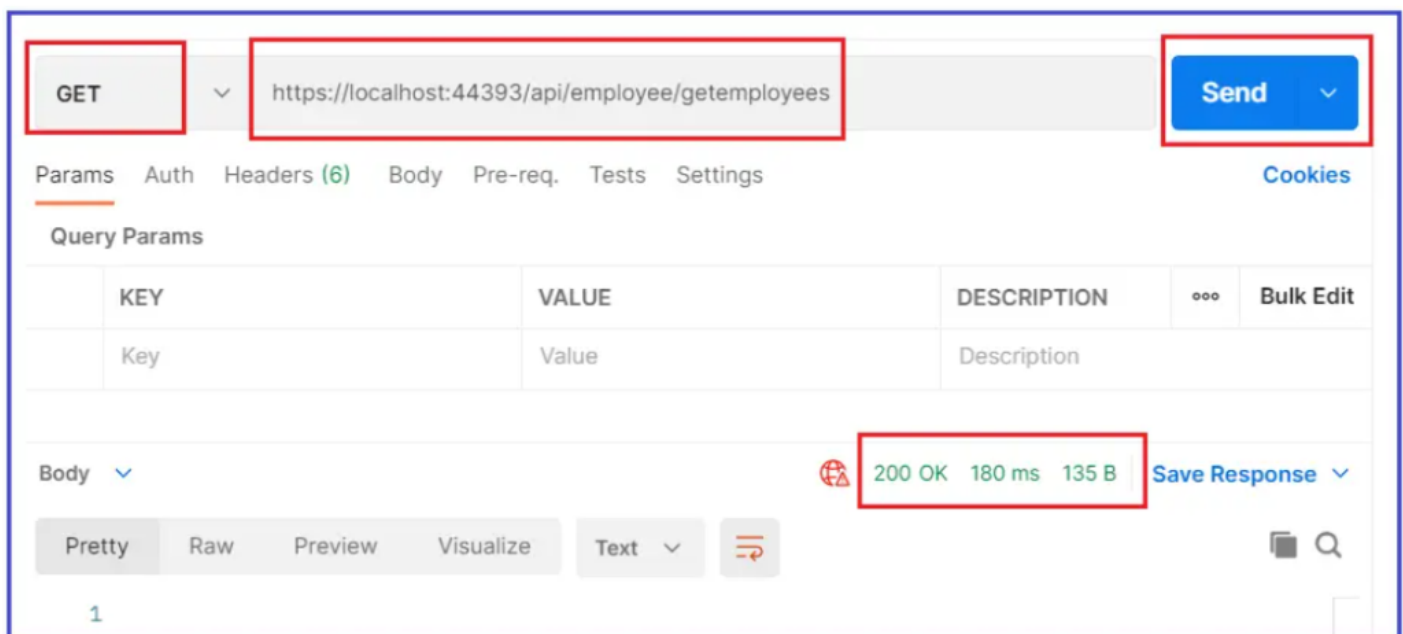
```
using Microsoft.AspNetCore.Mvc;
namespace ReturnTypeAndStatusCodes.Controllers
```

```

{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpGet("GetEmployees")]
        public IActionResult GetEmployees()
        {
            //Do Some Operation
            return Ok();
        }
    }
}

```

Run the application and issue a GET request to the URL **api/employee/getemployees** using Postman, as shown in the image below. Once you hit the Send button, you will get a **200 OK Status code** without any data in the response body.



Creating Employee Model:

We are going to use the following Employee model to return data from the controller action method. So, create a class file named **Employee.cs** and then copy and paste the following code into it:

```

namespace ReturnTyAndStatusCodes.Models
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Gender { get; set; }
    }
}

```

```

        public string City { get; set; }
        public int Age { get; set; }
        public string Department { get; set; }
    }
}

```

Returning 200 OK HTTP Status Code with Data:

In this case, we have to use the second overloaded version of the OK method, which takes an object as an input parameter. So, please modify the Employee Controller class as shown below. Here, we are returning the list of employees from the action method. As the OK method takes object type, we can pass any data. Here, I am passing the list of employees to the OK method.

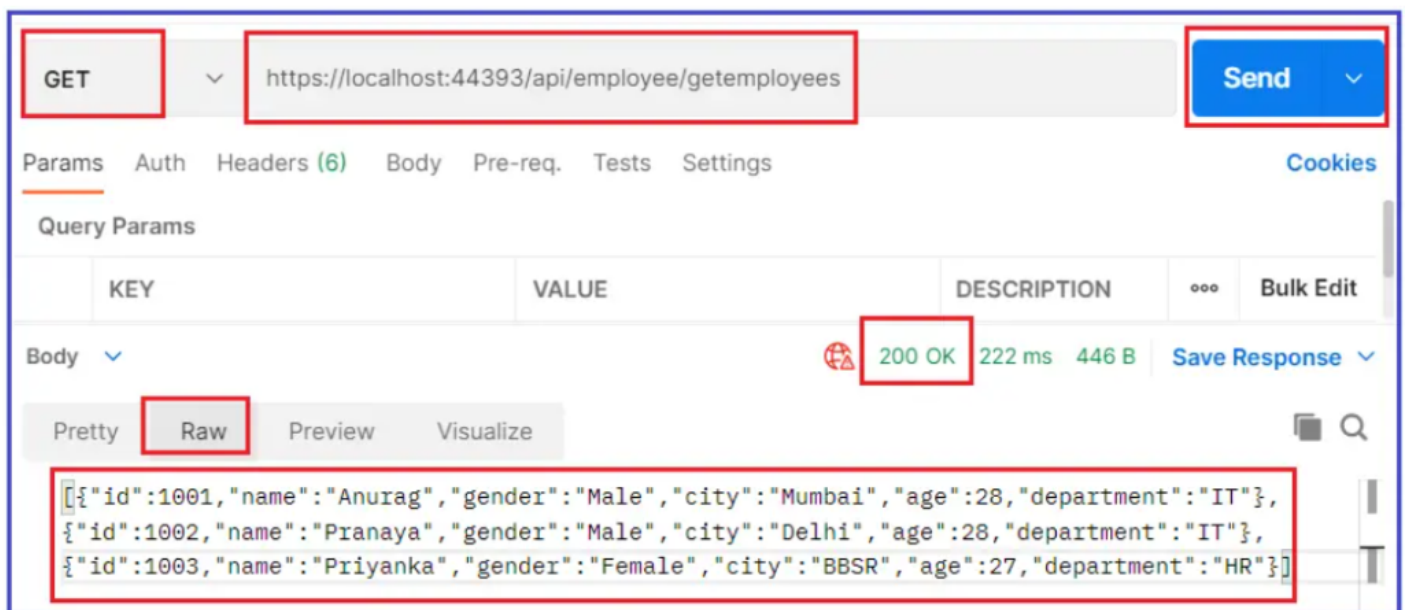
```

using ReturnTypesAndStatusCodes.Models;
using Microsoft.AspNetCore.Mvc;
namespace ReturnTypesAndStatusCodes.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpGet("GetEmployees")]
        public ActionResult<List<Employee>> GetEmployees()
        {
            var listEmployees = new List<Employee>()
            {
                new Employee(){ Id = 1001, Name = "Anurag", Age = 28, City = "Mumbai", Gender = "Male", Department = "IT" },
                new Employee(){ Id = 1002, Name = "Pranaya", Age = 28, City = "Delhi", Gender = "Male", Department = "IT" },
                new Employee(){ Id = 1003, Name = "Priyanka", Age = 27, City = "BBSR", Gender = "Female", Department = "HR" },
            };

            return Ok(listEmployees);
        }
    }
}

```

Now save the changes, run the application, and issue a GET request to the same URL **api/employee/getemployees** using Postman, as shown in the image below. Once you hit the Send button, you will get a **200 OK Status Code** with the employee data in the response body in JSON format.



The OK Method is very helpful for GET-type requests. For example, you can get data for all Employees, single employees, or employees based on some search parameters.

Manual Status Code Setting in ASP.NET Core Web API

Although it is not common for a 200 OK scenario, you can still manually set the response status code using the `StatusCode` method. This approach can be used to return any Status Code with or without data in ASP.NET Core Web API.

```
using ReturnTypesAndStatusCodes.Models;
using Microsoft.AspNetCore.Mvc;
namespace ReturnTypesAndStatusCodes.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpGet("GetEmployees")]
        public ActionResult<List<Employee>> GetEmployees()
        {
            var listEmployees = new List<Employee>()
            {
                new Employee(){ Id = 1001, Name = "Anurag", Age = 28, City = "Mumbai", Gender = "Male", Department = "IT" },
                new Employee(){ Id = 1002, Name = "Pranaya", Age = 28, City = "Delhi", Gender = "Male", Department = "IT" },
                new Employee(){ Id = 1003, Name = "Priyanka", Age = 27, City = "BBSR", Gender = "Female", Department = "HR"},
            };
        }
    }
}
```

```

        //return StatusCode(200); // Manually setting 200 OK without Data
        return StatusCode(200, listEmployees); // Manually setting 200 OK
with Data
    }
}
}

```

200 HTTP Status Code Real-Time Examples in ASP.NET Core Web API

In ASP.NET Core Web API, the HTTP 200 OK status code is typically used to indicate that a request has been successfully processed. For various operations like GET, POST, PUT, or DELETE, the API might return a 200 Status Code, possibly along with data. Please modify the Employee Controller as follows. Here, we are performing the CRUD operation.

```

using ReturnTypesAndStatusCodes.Models;
using Microsoft.AspNetCore.Mvc;
namespace ReturnTypesAndStatusCodes.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        //Data Source
        private static List<Employee> Employees = new List<Employee>
        {
            new Employee { Id = 1, Name = "Rakesh", Age = 25, City="BBSR", Gender
= "Male", Department = "IT" },
            new Employee { Id = 2, Name = "Hina", Age = 26, City="CTC", Gender =
"Female", Department = "HR" },
            new Employee { Id = 3, Name = "Suresh", Age = 27, City="RKL", Gender
= "Male", Department = "IT" },
        };

        //URL: GET /api/employee
        [HttpGet]
        public async Task<ActionResult> GetAllEmployees()
        {
            //Do Some IO Bound Operation
            //Delaying the Execution for 1000 MS
            await Task.Delay(TimeSpan.FromMilliseconds(1000));

            // 200 OK with all the employees in the response body
            return Ok(Employees);
        }

        //URL: GET /api/employee/1
    }
}

```

```

[HttpGet("{id}")]
public async Task<ActionResult<Employee>> GetEmployee(int id)
{
    //Do Some IO Bound Operation
    //Delaying the Execution for 1000 MS
    await Task.Delay(TimeSpan.FromMilliseconds(1000));

    var employee = Employees.FirstOrDefault(emp => emp.Id == id);
    if (employee == null)
    {
        // 404 Not Found if the employee does not exist
        return NotFound();
    }

    // 200 OK with the employee in the response body
    return Ok(employee);
}

//URL:POST /api/employee
[HttpPost]
public async Task<IActionResult> CreateOrUpdateEmployee(Employee
employee)
{
    //Do Some IO Bound Operation
    //Delaying the Execution for 1000 MS
    await Task.Delay(TimeSpan.FromMilliseconds(1000));

    var existingEmployee = Employees.FirstOrDefault(emp => emp.Id ==
employee.Id);
    if (existingEmployee != null)
    {
        // Update the existing employee
        existingEmployee.Name = employee.Name;
        existingEmployee.Age = employee.Age;
        existingEmployee.Gender = employee.Gender;
        employee.Department = employee.Department;

        // 200 OK with the updated employee
        return Ok(existingEmployee);
    }
    else
    {
        // Add a new employee
        employee.Id = Employees.Count() + 1;
        Employees.Add(employee);
        return CreatedAtAction(nameof(GetEmployee), new { id =
employee.Id }, employee);
    }
}

```



```

    }
}

//URL:PUT /api/employee/1
[HttpPut("{id}")]
public async Task<IActionResult> UpdateEmployee(int id, Employee
employee)
{
    //Do Some IO Bound Operation
    //Delaying the Execution for 1000 MS
    await Task.Delay(TimeSpan.FromMilliseconds(1000));

    if (id != employee.Id)
    {
        // 400 Bad Request if the ID does not match
        return BadRequest();
    }

    var existingEmployee = Employees.FirstOrDefault(emp => emp.Id ==
employee.Id);
    if (existingEmployee == null)
    {
        // 404 Not Found if the employee does not exist
        return NotFound();
    }

    // Update the existing employee
    existingEmployee.Name = employee.Name;
    existingEmployee.Age = employee.Age;
    existingEmployee.Gender = employee.Gender;
    employee.Department = employee.Department;

    // 200 OK with the updated employee
    return Ok(existingEmployee);
}

//URL:DELETE /api/employee/1
[HttpDelete("{id}")]
public ActionResult<Employee> DeleteEmployee(int id)
{
    var employee = Employees.FirstOrDefault(emp => emp.Id == id);
    if (employee == null)
    {
        // 404 Not Found if the employee does not exist
        return NotFound();
    }
}

```

```

        // Delete the employee
        Employees.Remove(employee);
        return Ok(employee); // 200 OK with the deleted employee's details
    }
}
}

```

Testing the APIs:

You can test the APIs in many ways, like using Postman, Fiddler, and Swagger. But .NET 8 provides the .http file, and using that .http file, we can also test the functionality very easily. This .http file name will be the same as your project name. So, open the .http file and copy and paste the following code. Please change the port number with the port number on which your application is running.

```
@MyFirstWebAPIProject_HostAddress = https://localhost:7166
```

```
GET {{MyFirstWebAPIProject_HostAddress}}/api/employee
```

```
Accept: application/json
```

```
###
```

```
### Get Employee with ID 1
```

```
GET {{MyFirstWebAPIProject_HostAddress}}/api/employee/1
```

```
Accept: application/json
```

```
###
```

```
### Create a New Employee
```

```
POST {{MyFirstWebAPIProject_HostAddress}}/api/employee
```

```
Content-Type: application/json
```

```
{
  "name": "Suresh",
  "gender": "Male",
  "age": 35,
  "city": "BBSR",
  "department": "HR"
}
```

```
###
```

```
### Update Employee with ID 1
```

```
PUT {{MyFirstWebAPIProject_HostAddress}}/api/employee/1
```

```
Content-Type: application/json
```

```
{
  "Id": 1,
  "name": "Suresh",

```

```
"gender": "Male",
"age": 35,
"city": "RKL",
"department": "HR"
}

###

### Delete Employee with ID 1
DELETE {{MyFirstWebAPIProject_HostAddress}}/api/employee/1
###
```

When Should We Return 200 OK HTTP Status Code in ASP.NET Core Web API?

In ASP.NET Core Web API, returning a 200 HTTP status code is appropriate in several scenarios, primarily indicating that a request has succeeded. The following are some of the specific situations in which returning a 200 OK status might be suitable:

- **GET Requests:** When handling GET requests that fetch data successfully, you should return a 200 OK status. The response typically includes the resource data requested by the client.
- **POST Requests with Resource Creation:** If a POST request creates a resource and you choose to return it in the response, 200 OK can be used, although 201 Created is more appropriate if you also provide a URI to the created resource in the response header. Use 200 OK if you are returning the newly created entity without a new URI.
- **PUT or PATCH Requests:** For update operations using PUT or PATCH, if the operation is successful and you wish to send the updated resource back to the client, 200 OK is appropriate. The response should include the updated data.
- **DELETE Requests:** When a DELETE request successfully removes a resource, you can use 200 OK if you want to return some information about the deleted resource or a confirmation message. However, 204 No Content is often used when no content is returned.
- **Complex Operations:** For actions that involve complex operations or business logic where the result isn't necessarily the creation, update, or deletion of a resource, you might use 200 OK to indicate successful processing along with relevant result data.

It's important to note that while 200 OK is suitable for many successful operations, other success status codes can provide more specific information about the request's outcome:

- **201 Created:** Specifically, for when a new resource has been successfully created.
- **202 Accepted:** Indicates that the request has been accepted for processing but has not been completed.
- **204 No Content:** Indicates that the server successfully processed the request but is not returning any content. Often used for DELETE operations or when PUT operations do not return any content.

In the next article, I will discuss [how to Return 201 HTTP Status Code in ASP.NET Core Web API](#) with Examples. In this article, I try to explain how to Return 200 OK HTTP Status Code from ASP.NET Core Web API with Examples. I hope you enjoy this article on “200 HTTP Status Codes in the ASP.NET Core Web API.”



Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

2 thoughts on “200 HTTP Status Code in ASP.NET Core Web API”



LARUE

MAY 21, 2024 AT 5:17 PM

Hi, Nice Post

[Reply](#)



SAM

MAY 22, 2024 AT 3:48 AM

I am actually thankful to the holder of this website who has shared this enormous article at at this time.

[Reply](#)

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information